



数値計算法 第11回

関数近似 (2)

濱本 雄治¹

情報工学部 情報通信工学科

2026 年 6 月 29 日

¹hamamoto@c.oka-pu.ac.jp

前回の復習：最小二乗近似の考え方



▶ 標本点の最も合う関数を求める

- ▷ $y = f(x)$ を満たす m 個の標本点 $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$
- ▷ 標本点の近傍を通過する n 次多項式で $f(x)$ を近似

$$p_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n \quad (\text{最小二乗近似多項式})$$

- ▷ 誤差の二乗和 $Q(\{a_k\})$

$$Q(\{a_k\}) = \sum_{k=1}^m |p_n(x_k) - y_k|^2$$

- ▷ $Q(\{a_k\})$ の最小化から係数 $a_0, a_1, \dots, a_n$ を決定

$$\frac{\partial Q}{\partial a_1} = \frac{\partial Q}{\partial a_2} = \dots = \frac{\partial Q}{\partial a_n} = 0$$

前回の復習：最小二乗近似の手順



▶ n 次多項式による最小二乗近似

▷ $Q(\{a_k\})$ の二乗和の最小化から得られる正規方程式

$$\begin{pmatrix} S_{x^0} & S_{x^1} & \cdots & S_{x^n} \\ S_{x^1} & S_{x^2} & \cdots & S_{x^{n+1}} \\ \vdots & \vdots & \ddots & \vdots \\ S_{x^n} & S_{x^{n+1}} & \cdots & S_{x^{2n}} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} S_{x^0 y} \\ S_{x^1 y} \\ \vdots \\ S_{x^n y} \end{pmatrix}$$

▷ m 個の標本点 $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$ から行列要素を計算

$$S_{x^\ell} = \sum_{k=1}^m x_k^\ell, \quad S_{x^\ell y} = \sum_{k=1}^m x_k^\ell y_k$$

▷ Cramer の公式で正規方程式を解き、係数 $a_0, a_1, a_2, \dots, a_n$ を決定

前回の小テストの解説



データ点 $(0, 0), (1, 1), (2, 4), (3, 8)$ が満たす関数 $y = f(x)$ を一次式 $p(x) = a + bx$ で近似するとき、最小二乗法で定数 a, b を求めよ。

$$S_{x^0} = 4, \quad S_{x^1} = 0 + 1 + 2 + 3 = 6, \quad S_{x^2} = 0 + 1 + 4 + 9 = 14,$$

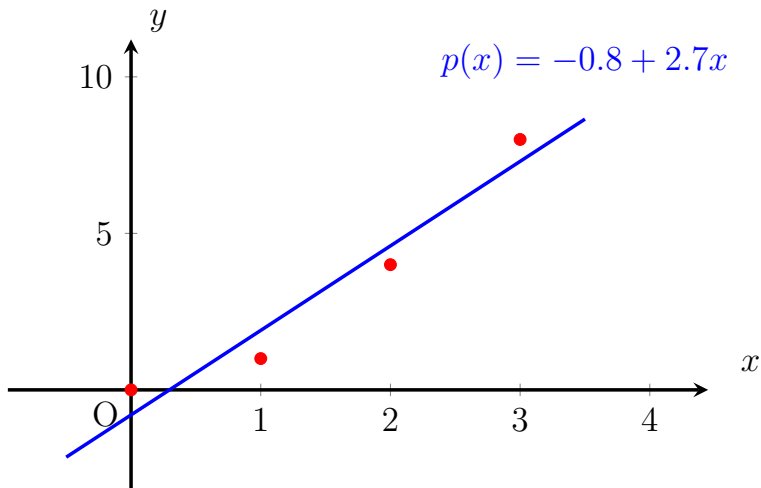
$$S_{x^0y} = 0 + 1 + 4 + 8 = 13, \quad S_{x^1y} = 0 + 1 + 8 + 24 = 33,$$

$$\begin{pmatrix} 4 & 6 \\ 6 & 14 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 13 \\ 33 \end{pmatrix}$$

$$a = \frac{13 \times 14 - 6 \times 33}{4 \times 14 - 6 \times 6} = -\frac{4}{5}, \quad b = \frac{4 \times 33 - 13 \times 6}{4 \times 14 - 6 \times 6} = \frac{27}{10},$$

$$\therefore p(x) = -0.8 + 2.7x$$

最小二乗1次多項式と標本点の比較



第11回の目標



▶ Lagrange 補間による関数近似を理解する

- ▷ $n + 1$ 個の標本点 $(x_0, y_0), \dots, (x_n, y_n)$
- ▷ すべての標本点を通過する n 次補間多項式をどう求めるか

$$p_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

- ▷ Lagrange 補間では計算コストがどう改善されるか

▶ すべての標本点を通過する関数を求める

- ▷ $y = f(x)$ を満たす m 個の標本点 $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$
- ▷ 標本点を通過する n 次多項式で $f(x)$ を近似

$$p_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n \quad (n \text{ 次補間多項式})$$

- ▷ $n + 1$ 個の標本点を通過する n 次多項式は一意に決まる

例: 一次関数 $y = a + bx$ が 2 点 $(x_0, y_0), (x_1, y_1)$ を通過するとき

$$\begin{cases} a + bx_0 = y_0 \\ a + bx_1 = y_1 \end{cases} \Leftrightarrow \begin{pmatrix} 1 & x_0 \\ 1 & x_1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \end{pmatrix}$$

- ▷ 未知数が n 個の連立一次方程式の計算コスト $\sim \mathcal{O}(n^3)$

→ 効率的に補間多項式を導出する方法が必要



Lagrange 補間の考え方

▶ 連立一次方程式を解かずに補間多項式を求める

▷ $x = x_0, x_1, x_2, \dots, x_n$ のとき次の $n + 1$ 次多項式はゼロ

$$L(x) = (x - x_0)(x - x_1)(x - x_2) \cdots (x - x_n)$$

▷ $L(x)$ から因子 $x - x_i$ を取り除いた n 次多項式

$$\begin{aligned} L_i(x) &= (x - x_0) \cdots (x - x_{i-1}) \cancel{(x - x_i)} (x - x_{i+1}) \cdots (x - x_n) \\ &= \begin{cases} L_i(x_i) \neq 0 & (x = x_i), \\ 0 & (x = x_0, \dots, x_{i-1}, x_{i+1}, \dots, x_n) \end{cases} \end{aligned}$$

▷ $n + 1$ 個の標本点 $(x_0, y_0), \dots, (x_n, y_n)$ を通る n 次多項式

$$p_n(x) = \sum_{i=0}^n c_i L_i(x) = c_0 L_0(x) + \cdots + c_n L_n(x) \quad (\text{Lagrange 補間})$$

▷ $p_n(x_j) = c_j L_j(x_j)$ から第 j 項の係数 $c_j = \frac{p_n(x_j)}{L_j(x_j)}$ が求まる

Lagrange 補間の例



▶ $n = 1$ のとき

▷ 2 点 $(x_0, y_0), (x_1, y_1)$ を通る 1 次の Lagrange 補間多項式

$$y = a \underbrace{(x - x_1)}_{x-x_0 \text{ 以外}} + b \underbrace{(x - x_0)}_{x-x_1 \text{ 以外}}$$

▷ (x, y) に $(x_0, y_0), (x_1, y_1)$ を代入すると

$$y_0 = a(x_0 - x_1) + 0, \quad \therefore a = \frac{y_0}{x_0 - x_1},$$

$$y_1 = 0 + b(x_1 - x_0), \quad \therefore b = \frac{y_1}{x_1 - x_0}$$

▷ したがって求める一次多項式は

$$y = y_0 \frac{x - x_1}{x_0 - x_1} + y_1 \frac{x - x_0}{x_1 - x_0}$$

Lagrange 補間の例 (続き)



▶ $n = 2$ のとき

▷ 3 点 (x_0, y_0) , (x_1, y_1) , (x_2, y_2) を通る 2 次の Lagrange 補間多項式

$$y = a \underbrace{(x - x_1)(x - x_2)}_{x-x_0 \text{ 以外}} + b \underbrace{(x - x_0)(x - x_2)}_{x-x_1 \text{ 以外}} + c \underbrace{(x - x_0)(x - x_1)}_{x-x_2 \text{ 以外}}$$

▷ (x, y) に (x_0, y_0) , (x_1, y_1) , (x_2, y_2) を代入すると

$$y_0 = a(x_0 - x_1)(x_0 - x_2) + 0 + 0, \quad \therefore a = \frac{y_0}{(x_0 - x_1)(x_0 - x_2)},$$

$$y_1 = 0 + b(x_1 - x_0)(x_1 - x_2) + 0, \quad \therefore b = \frac{y_1}{(x_1 - x_0)(x_1 - x_2)},$$

$$y_2 = 0 + 0 + c(x_2 - x_0)(x_2 - x_1), \quad \therefore c = \frac{y_2}{(x_2 - x_0)(x_2 - x_1)}$$

▷ したがって求める二次多項式は

$$y = y_0 \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} + y_1 \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} + y_2 \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)}$$

Lagrange 補間の例 (続き)



▶ $n = 3$ のとき

▷ 4 点 $(x_0, y_0), (x_1, y_1), (x_2, y_2), (x_3, y_3)$ を通る 3 次の Lagrange 補間多項式は、 $n = 1, 2$ の場合の類推から

$$\begin{aligned} y = & y_0 \frac{\overbrace{(x - x_1)(x - x_2)(x - x_3)}^{x-x_0 \text{ 以外の積}}}{\underbrace{(x_0 - x_1)(x_0 - x_2)(x_0 - x_3)}_{\text{分子に } x=x_0 \text{ を代入}}} + y_1 \frac{\overbrace{(x - x_0)(x - x_2)(x - x_3)}^{x-x_1 \text{ 以外の積}}}{\underbrace{(x_1 - x_0)(x_1 - x_2)(x_1 - x_3)}_{\text{分子に } x=x_1 \text{ を代入}}} \\ & + y_2 \frac{\overbrace{(x - x_0)(x - x_1)(x - x_3)}^{x-x_2 \text{ 以外の積}}}{\underbrace{(x_2 - x_0)(x_2 - x_1)(x_2 - x_3)}_{\text{分子に } x=x_2 \text{ を代入}}} + y_3 \frac{\overbrace{(x - x_0)(x - x_1)(x - x_2)}^{x-x_3 \text{ 以外の積}}}{\underbrace{(x_3 - x_0)(x_3 - x_1)(x_3 - x_2)}_{\text{分子に } x=x_3 \text{ を代入}}} \end{aligned}$$

Lagrange 補間多項式の一般形



▶ 一般の n の場合

- ▶ $(x - x_0)(x - x_1) \cdots (x - x_n)$ から $x - x_i$ を取り除いた n 次多項式

$$L_i(x) = (x - x_0) \cdots (x - x_{i-1}) \cancel{(x - x_i)} (x - x_{i+1}) \cdots (x - x_n) = \prod_{j(\neq i)} (x - x_j)$$

- ▶ $n + 1$ 個の点 $(x_0, y_0), \dots, (x_n, y_n)$ を通る n 次の補間多項式

$$p_n(x) = \sum_{i=0}^n y_i \frac{L_i(x)}{L_i(x_i)} = \sum_{i=0}^n y_i \prod_{j(\neq i)} \frac{x - x_j}{x_i - x_j}$$

▶ 計算コスト

- ▶ $L_i(x)/L_i(x_i)$ の計算 $\sim \mathcal{O}(n)$
- ▶ Lagrange 補間全体 $\sim \mathcal{O}(n^2)$ $\ll$ 連立一次方程式 $\simeq \mathcal{O}(n^3)$

Lagrange 補間多項式の例



▶ 4 個の標本点 (1, 1), (2, 2), (4, 3), (8, 4)

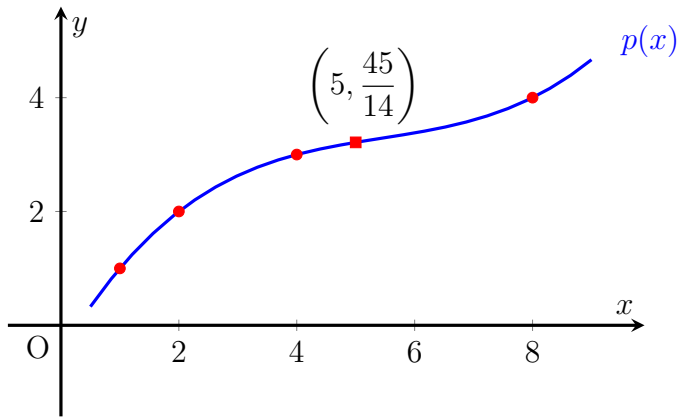
▷ 3 次の Lagrange 補間多項式

$$\begin{aligned} p_3(x) &= 1 \frac{(x-2)(x-4)(x-8)}{(1-2)(1-4)(1-8)} + 2 \frac{(x-1)(x-4)(x-8)}{(2-1)(2-4)(2-8)} \\ &\quad + 3 \frac{(x-1)(x-2)(x-8)}{(4-1)(4-2)(4-8)} + 4 \frac{(x-1)(x-2)(x-4)}{(8-1)(8-2)(8-4)} \\ &= -\frac{1}{21}(x-2)(x-4)(x-8) + \frac{1}{6}(x-1)(x-4)(x-8) \\ &\quad - \frac{1}{8}(x-1)(x-2)(x-8) + \frac{1}{42}(x-1)(x-2)(x-4) \end{aligned}$$

▷ $x = 5$ における関数値

$$\begin{aligned} p_3(5) &= -\frac{3 \cdot 1 \cdot (-3)}{21} + \frac{4 \cdot 1 \cdot (-3)}{6} - \frac{4 \cdot 3 \cdot (-3)}{8} + \frac{4 \cdot 3 \cdot 1}{42} \\ &= \frac{3}{7} - 2 + \frac{9}{2} + \frac{2}{7} = \frac{45}{14} \end{aligned}$$

Lagrange 補間多項式と標本点、予測点の比較



Lagrange 補間の応用: 逆補間



- ▶ $y = f(x)$ を満たす標本点を与えられたとき

通常の間補間

任意の x に対して $y = f(x)$
の近似値を求める



逆補間

任意の y を与える $x = f^{-1}(y)$
の近似値を求める

- ▶ 逆補間の手順

1. x, y を入れ替えた Lagrange 補間多項式を求める

$$p_n(y) = \sum_{i=0}^n x_i \frac{L_i(y)}{L_i(y_i)}$$

2. 任意の y を与える x の近似値を $x = f^{-1}(y) \simeq p_n(y)$ で求める

逆補間の例



▶ $\sqrt{2}$ の近似値

- ▶ $y = f(x) = x^2$ とすると $x = f^{-1}(y) = \sqrt{y}$
- ▶ $1.4 < \sqrt{2} < 1.5$ は既知として $x = \sqrt{2}$ 近傍の標本点を採用

x	1.3	1.4	1.5	1.6
y	1.69	1.96	2.25	2.56

- ▶ x, y を入れ替えた 3 次の Lagrange 補間多項式は

$$\begin{aligned} p_3(y) = & 1.3 \frac{(y - 1.96)(y - 2.25)(y - 2.56)}{(1.69 - 1.96)(1.69 - 2.25)(1.69 - 2.56)} \\ & + 1.4 \frac{(y - 1.69)(y - 2.25)(y - 2.56)}{(1.96 - 1.69)(1.96 - 2.25)(1.96 - 2.56)} \\ & + 1.5 \frac{(y - 1.69)(y - 1.96)(y - 2.56)}{(2.25 - 1.69)(2.25 - 1.96)(2.25 - 2.56)} \\ & + 1.6 \frac{(y - 1.69)(y - 1.96)(y - 2.25)}{(2.56 - 1.69)(2.56 - 1.96)(2.56 - 2.25)} \end{aligned}$$

計算結果



► lagrange.c

```
#include <stdio.h>

int main(){
    int np = 3;
    double xmin = 1.3;
    double xmax = 1.6;
    double dx = (xmax - xmin) / np;
    double xp[np+1];
    double yp[np+1];
    for(int i=0; i<=np; ++i){
        double x = xmin + dx * i;
        xp[i] = x;
        yp[i] = x * x;
    }
```

```
double x = 0.0;
double y = 2.0;
for(int j=0; j<=np; ++j){
    double prod = xp[j];
    for(int k=0; k<=np; ++k){
        if(k==j)
            continue;
        prod*=(y-yp[k])/(yp[j]-yp[k]);
    }
    x += prod;
}
printf("%f %f\n", x, y);
}
```

```
$/a.out
1.414219 2.000000
```

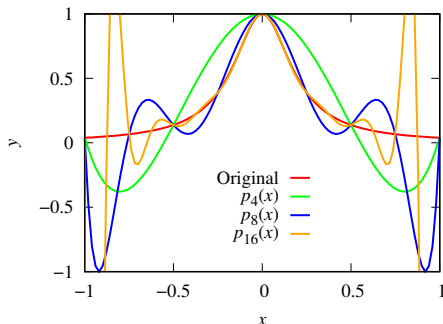
Runge の現象



- ▶ 補間点の数を増やすほど補間精度が悪化することがある
 - ▷ Runge の関数

$$f(x) = \frac{1}{1 + 25x^2}$$

- ▷ 区間 $[-1, 1]$ で Lagrange 補間すると両端で振動



補足: Runge の現象を計算するプログラム



▶ runge.c

```
#include <stdio.h>
int main(){
    int np = 16;
    double xmin = -1.0;
    double xmax = 1.0;
    double dx = (xmax - xmin) / np;
    double xp[np+1];
    double yp[np+1];
    for(int i=0; i<=np; ++i){
        double x = xmin + dx * i;
        xp[i] = x;
        yp[i] = 1.0 / (1.0 + 25*x*x);
    }
    int nx = np * 10;
    dx = (xmax - xmin) / nx;
```

```
    for(int i=0; i<=nx; ++i){
        double x = xmin + dx * i;
        double y = 0.0;
        for(int j=0; j<=np; ++j){
            double prod = yp[j];
            for(int k=0; k<=np; ++k){
                if(k==j)
                    continue;
                prod*=(x-xp[k])/(xp[j]-xp[k]);
            }
            y += prod;
        }
        printf("%f %f\n", x, y);
    }
}
```

第11回のまとめ



▶ 補間による関数近似

- ▷ すべての標本点を通過する関数を求める
- ▷ $n + 1$ 個の標本点を通過する n 次多項式は一意に決まる

▶ Lagrange 補間

- ▷ $n + 1$ 個の標本点 $(x_0, y_0), \dots, (x_n, y_n)$ を通過する n 次補間多項式

$$p_n(x) = \sum_{i=0}^n y_i \prod_{j(\neq i)} \frac{x - x_j}{x_i - x_j}$$

- ▷ 計算コスト $\sim \mathcal{O}(n^2)$
- ▷ 補間点を増やすと精度が低下することがある (Runge の現象)